SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA1707**

COURSE OUTCOME COVERED

CO2: *Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)*

Assessment Tool Weightage: 35%

Duration: 1 Hour

QUESTIONS: 5 **Total Marks:50**

Annexure A

Scenario Based Assignment

Code of Conduct:

I ***S. Lakshmi Priya (192524284)*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: **S. Lakshmi Priya**

Annexure A

Scenario Based Assignment

1. Scenario : A government deploys AI-powered drones to deliver emergency medicines in a flood-affected region. The environment is highly dynamic—roads may suddenly become inaccessible, and weather conditions affect travel cost. The drone has access to: A heuristic estimate (straight-line distance), Partial real-time updates about blocked paths and Variable movement costs depending on terrain and weather. However, due to urgency, the drone must quickly decide routes with minimum delay and risk.

Tasks:

i. Explain how Greedy Best-First Search would behave in this scenario and discuss its strengths and weaknesses under uncertainty

ii. Explain how A* would handle the same situation, including how it balances cost and heuristic

iii. Analyze the impact of heuristic accuracy on both algorithms

iv. Discuss how dynamic changes (blocked paths) affect search performance

v. Recommend the most suitable algorithm and justify your decision considering real-time constraints, optimality, and safety

Answer 1: Emergency Medicine Drone – Greedy Best-First Search vs A*

(i) Behaviour of Greedy Best-First Search (GBFS)

GBFS expands the node that appears closest to the goal, i.e. it orders the frontier only by f(n) = h(n), where h(n) is the straight-line distance to the delivery point. The drone therefore always flies toward the node that looks nearest to the flood-affected village and ignores the terrain/weather cost already spent (g(n)).

- **Strengths:** very fast, expands few nodes, low memory, produces a first route almost immediately – useful when urgency is high and the map changes quickly.
- **Weaknesses under uncertainty:** (a) not optimal – it can pick a short-looking path through a storm zone with very high movement cost; (b) not complete in graphs with loops/dead ends unless a closed list is kept; (c) it is easily trapped by a blocked road or river that lies directly between drone and target, because the straight-line heuristic keeps pulling it toward the obstacle; (d) it gives no guarantee on risk or delay; (e) worst case time and space are O(b^m).

(ii) How A* handles the situation

A* uses f(n) = g(n) + h(n). g(n) is the actual accumulated cost from the base to node n (distance multiplied by terrain/weather factors, risk penalties), and h(n) is the straight-line estimate of the remaining cost. The g(n) term makes the drone avoid cheap-looking but costly (stormy, rough-terrain) segments; the h(n) term keeps the search directed toward the goal rather than spreading in all directions like Uniform-Cost Search.

- If h(n) is admissible (never overestimates) and consistent, A* is complete and returns the least-cost route. Straight-line distance is admissible because no real route is shorter than a straight line, even when weather multiplies the actual cost.

- Balance: when h dominates, A* behaves like GBFS (fast, risky); when g dominates, it behaves like UCS (safe but slow). Weighted A* with f = g + w·h (w slightly above 1, e.g. 1.2–1.5) can be used to trade a small loss in optimality for speed.

Worked example (f = g + h)

Node	g(n) cost so far	h(n) straight-line	f = g + h	GBFS picks (h only)
A (clear, long)	4	6	10	–
B (storm zone, short)	9	2	11	B (lowest h = 2)
C (via river road)	5	5	10	–

GBFS chooses B because h = 2 is the smallest, but its real cost is high and risky. A* compares f-values and chooses A or C (f = 10), which is cheaper overall and safer. Ties are broken in favour of lower h.

(iii) Impact of heuristic accuracy

- **Perfect heuristic h = h*:** A* expands only nodes on the optimal path; GBFS also goes straight to the goal.
- **Underestimating (admissible but weak, e.g. h = 0):** A* stays optimal but expands more nodes (becomes UCS); GBFS degenerates into random/uninformed search.
- **Overestimating (inadmissible):** A* may skip the optimal route and return a sub-optimal one; GBFS becomes over-confident and rushes into costly regions.
- **Inaccurate in flood conditions:** straight-line distance ignores blocked roads, wind and rain, so it can mislead both. A* is more robust because g(n) corrects errors with real cost; GBFS has no such correction and is the most sensitive to heuristic error. Better heuristics (distance × weather factor, terrain-aware map) improve both.

(iv) Effect of dynamic changes (blocked paths)

- A newly blocked path invalidates the plan. GBFS can re-plan very quickly (small search) but may keep re-selecting routes toward the same blocked area and can oscillate. A* re-plans more slowly but with better quality because blocked edges become infinite cost in g(n).
- Practical solution: use incremental/replanning variants – D* Lite or Lifelong Planning A* (LPA*) – which reuse earlier search results and repair only the affected part of the path when partial real-time updates arrive.
- Frequent changes increase computation, cause plan instability and risk of dead ends; the drone should re-plan at fixed intervals or when an update affects its current route, and keep an alternate (backup) route.

(v) Recommendation

Recommended: A* (preferably in its replanning form D* Lite / Weighted A*). Justification:

- **Real-time:** with a good heuristic and a modest weight w, A* expands far fewer nodes than UCS and can reply within the urgent time limit; GBFS is faster but unreliable.
- **Optimality:** A* with an admissible heuristic guarantees minimum-cost (minimum delay/risk) routes; GBFS does not.

- **Safety:** because g(n) includes weather, terrain and risk costs, dangerous areas are penalised and blocked paths are excluded; GBFS can lead the drone into hazards.

GBFS may be used only as a quick fallback when the time budget is almost exhausted. Therefore A* with replanning is the best choice for medicine delivery.

2. **Scenario**: A smart city implements an AI system to optimize traffic signal timings across hundreds of intersections. The objective is to minimize Total waiting time, Fuel consumption and Traffic congestion. The system must continuously adjust signals based on traffic flow. However the search space is extremely large, traffic patterns change dynamically and the system may get stuck in locally optimal solutions

Tasks. Formulate this as an optimization problem and explain why traditional search is inefficient

i. Explain how Hill Climbing would work in this scenario and identify its limitations

ii. Discuss issues like local maxima, plateaus, and ridges with real-world interpretation

iii. Explain how Simulated Annealing or Genetic Algorithms can overcome these limitations

iv. Recommend the best approach and justify your choice based on scalability and adaptability

Answer 2: Smart City Traffic Signal Optimisation

(i) Formulation as an optimisation problem

- **State:** the vector of signal timings for all N intersections (green time, cycle length, offset/phase order) together with current traffic conditions.
- **Initial state:** current/fixed-time signal plan.
- **Actions / neighbours:** change the green time, cycle length or offset of one or a few intersections by a small amount.
- **Objective function (to minimise):** F = w1·(Total waiting time) + w2·(Fuel consumption) + w3·(Congestion/queue length), with weights w1 + w2 + w3 = 1 reflecting city priorities.
- **Constraints:** minimum and maximum green times, pedestrian crossing time, emergency-vehicle priority, phase safety.

Why traditional (BFS/DFS/A*) search is inefficient: if each of N = 100 intersections has just 10 timing options, the space is 10^100 states, far beyond enumeration. The solution path does not matter – only the final good configuration – so path-based search wastes memory. Traffic changes continuously, so a stored optimal plan quickly becomes obsolete. No exact heuristic or goal test exists, as the "goal" is a minimum of a noisy function. Hence local-search / metaheuristic methods are preferred.

(ii) Hill Climbing and its limitations

Hill Climbing starts from the current timing plan, generates neighbouring plans (adjust one signal), evaluates the objective and moves to the best neighbour that improves the objective (steepest-ascent), stopping when no neighbour is better. It uses constant memory and is quick, so it fits large problems and can be run continuously as traffic changes.

- Limitations: gets stuck in local optima, cannot cross plateaus or ridges, depends on the starting point, is deterministic (so repeated runs give the same stuck result) and gives no guarantee of the global best.

(iii) Local maxima, plateaus and ridges – real-world meaning

Problem	Technical meaning	Traffic interpretation
Local maximum	A state better than all neighbours but worse than the global best	Timings that look best for the central junctions, but any single change worsens traffic; a totally different coordination plan (green wave) would be much better
Plateau	Flat region where neighbours have equal objective values	Small timing changes (±1 s) give no measurable change in waiting time, so the algorithm cannot tell which direction to move
Ridge	Narrow path of improvement not aligned with single-step moves	Improvement needs several coordinated signals to change together (offsets of consecutive intersections); changing one signal alone makes things worse

(iv) Simulated Annealing and Genetic Algorithms

Simulated Annealing (SA): picks a random neighbour; if it is better it is accepted, and if it is worse it is accepted with probability $P = e^{(-\Delta E/T)}$, where ΔE is the increase in cost and T the temperature. T starts high (explores widely, escapes local optima and plateaus) and is gradually reduced by a cooling schedule (e.g. $T = \alpha \cdot T$, $\alpha = 0.95$), converging towards a good solution. It needs little memory and is guaranteed to converge to the global optimum under a sufficiently slow cooling schedule.

Genetic Algorithm (GA): maintains a population of candidate timing plans (chromosome = timings of all intersections). Each generation applies fitness evaluation (1/total cost), selection of the fittest, crossover (combining the timings of two good plans) and mutation (random timing change). Crossover can change many intersections at once, helping with ridges, and the population explores many regions in parallel, avoiding being trapped in one local maximum. It is also easy to parallelise across city zones.

(v) Recommended approach

Recommended: a hybrid – Genetic Algorithm (global exploration, offline/periodic planning over city zones) combined with Simulated Annealing / Hill Climbing (fast local fine-tuning online).

- **Scalability:** GA is parallel and works on huge spaces; the city can be divided into zones optimised independently and then coordinated. SA/hill climbing need very little memory.
- **Adaptability:** the previous best solution is used as the starting population/state when traffic changes, so only small re-optimisation is needed in real time; random restarts and mutation allow adaptation to new patterns (events, accidents).
- **Quality:** avoids the local-maximum trap of plain hill climbing. If only one method is allowed, Simulated Annealing is the simplest and most effective choice for continuous online adjustment.

3. **Scenario:** An autonomous rover is deployed on Mars to explore unknown terrain. It must: navigate safely to collect samples, avoid hazards (craters, rocks), operate with limited sensing capability and make decisions without a complete map. The rover updates its knowledge as it explores, making this a real-time decision-making problem.

Tasks:

i. Explain why this problem requires an online search agent instead of offline search

ii. Analyze the challenges of partial observability and dynamic environments

iii. Describe how the rover builds and updates its internal model

iv. Compare online search with uninformed and informed search strategies

v. Justify the most suitable approach considering uncertainty, adaptability, and safety

Answer 3: Mars Rover – Online Search Agent

(i) Why an online search agent is required

An offline agent computes the whole plan before acting, which needs a complete, known model of the environment (map, actions, costs). On Mars the terrain is unknown, there is no complete map, and sensing is limited. The rover can only discover the outcome of an action by executing it. Online search interleaves computation and action: the agent plans one step, acts, senses, updates its model and plans again. It is also necessary because communication delay with Earth (4–24 minutes one way) prevents remote control, and the environment (dust, rock slides, new hazards) may change. Offline planning would waste time on an inaccurate plan or be unsafe, as exploring in "imagination" cannot reveal unseen craters. Online search is the standard approach for exploration problems with unknown states and actions.

(ii) Challenges of partial observability and dynamic environments

- **Limited sensor range/noise:** the rover sees only nearby terrain, so it must act on a belief about unseen areas; sensor error and wheel slippage cause position uncertainty.
- **Unknown consequences of actions:** a slope may be steeper or sandier than predicted; irreversible actions (falling into a crater, getting stuck) are dangerous – the environment is not safely explorable unless every state can be escaped from.
- **Local minima/dead ends:** the rover may enter a dead end and must backtrack, which costs energy and time.
- **Dynamic environment:** dust storms, shifting sand, changing light/temperature and new obstacles make the learned map outdated; the plan must be revised continuously.
- **Resource limits:** battery, computing power and communication restrict how much can be explored or computed.

(iii) Building and updating the internal model

- **Start:** an empty or partial map (grid/occupancy map) with all cells marked "unknown"; the current position is recorded.
- **Sense:** sensors (camera, LIDAR, IMU, wheel odometry) detect neighbouring cells – free, rock, crater, slope.
- **Update:** the map is updated for each observation (e.g. probabilistic occupancy grid with Bayesian update), and for each state the rover stores the results of actions tried:

RESULT[s, a] = s′, and the visited/unexplored-action list, cost estimates H(s) (as in LRTA*).

- **Localise and map:** SLAM (Simultaneous Localisation and Mapping) keeps the position estimate consistent with the map, and a belief state represents uncertainty about what is not observed.
- **Replan:** after each update the rover selects the next action using its current model (e.g. move toward the least-cost unexplored frontier), then repeats the sense–update–act cycle. When it hits a dead end it uses the stored model to backtrack to the nearest state with unexplored actions.

(iv) Comparison: online search vs uninformed and informed search

Aspect	**Uninformed (BFS/DFS/UCS)**	**Informed (Greedy, A*)**	**Online search (online DFS, LRTA*, D*)**
Knowledge needed	Full model of environment	Full model + heuristic	No complete model; learns while moving
Planning	Offline (whole path first)	Offline	Interleaved planning and acting
Use of heuristic	None	Yes, guides search	Heuristic updated by experience (LRTA*)
Handles unknown / changing terrain	No	No (needs known map)	Yes
Safety / reversibility	Assumes simulated actions are free	Same	Real actions cost energy; must avoid irreversible moves
Optimality	BFS/UCS optimal on known map	A* optimal if h admissible	Not guaranteed; competitive ratio used
Suitability for Mars	Poor	Poor on its own	Best

Uninformed and informed algorithms work only if the entire state space is known. Online agents may use informed ideas (a heuristic like straight-line distance to a sample site), but they must also learn the map while moving.

(v) Most suitable approach

Recommended: an online informed search agent – LRTA* / D* Lite based planning on a continuously updated map (SLAM), with a safety layer.

- **Uncertainty:** belief state/probabilistic map and re-planning after each observation handle partial observability.
- **Adaptability:** D* Lite repairs the path efficiently when new obstacles are discovered; LRTA* learns better heuristic values with experience and avoids infinite loops.
- **Safety:** treat unknown or steep cells as high-cost or forbidden, keep a path back to the last safe/charging location, prefer reversible moves, use conservative speed and obstacle margins, and trigger safe-mode when confidence is low.
- Combine with a high-level goal selector (nearest valuable sample vs energy remaining) so exploration is purposeful, not random.

4. **Scenario:** A university is designing an AI system to automatically generate exam timetables. The system must satisfy multiple constraints: No student should have overlapping exams, Limited number of exam halls, Certain subjects must be scheduled before others, Faculty availability constraints, Special accommodations for some students, The complexity increases as the number of courses and students grows.

Tasks :

Formulate the problem as a Constraint Satisfaction Problem (CSP)

i. Clearly define variables, domains, and constraints with explanation

ii. Explain how backtracking search works in this scenario

iii. Discuss how constraint propagation (e.g., forward checking) improves efficiency

iv. Analyze real-world challenges and justify the best strategy for scalability

Answer 4: University Exam Timetabling as a CSP

(i) and (ii) CSP formulation: variables, domains and constraints

A CSP is defined by a triple (X, D, C). Here:

- **Variables (X):** one variable for each exam, E1, E2, …, En (one per course). Optionally two variables per exam: Time(Ei) and Hall(Ei).
- **Domains (D):** Time(Ei) ∈ {all available (day, slot) pairs, e.g. 10 days × 2 sessions = 20 slots}, Hall(Ei) ∈ {H1, H2, …, Hk} (halls with enough capacity). The domain may be reduced in advance for faculty availability and special accommodations.
- **Constraints (C):**

Type	Constraint	Explanation
Binary (hard)	If courses Ci and Cj share a student, Time(Ei) ≠ Time(Ej)	No student has overlapping exams (conflict graph colouring)
Resource (hard)	If Time(Ei) = Time(Ej) then Hall(Ei) ≠ Hall(Ej); number of exams in a slot ≤ number of halls; students in a hall ≤ capacity	Limited exam halls
Precedence (hard)	Time(Ei) < Time(Ej) if Ei must be before Ej	Some subjects scheduled before others
Unary (hard)	Time(Ei) ∈ slots when invigilator/faculty is available	Faculty availability
Unary / special (hard)	Students with accommodations get a separate/accessible hall, extra time (longer slot), no back-to-back exams	Special accommodations
Soft (preference)	Minimise a student having 2+ exams on one day; spread exams evenly	Fairness; used as an optimisation objective

Because all hard constraints involve at most two variables (or a global limit), the problem is similar to graph colouring: courses are nodes, an edge joins courses with common students, and "colours" are time slots. It is NP-hard, so complexity grows rapidly with courses and students.

(iii) How backtracking search works

Backtracking is depth-first search that assigns one variable at a time and checks consistency:

- 1. Choose an unassigned exam (e.g. by Minimum Remaining Values – MRV).
- 2. Try a value (slot, hall) from its domain (e.g. by Least Constraining Value).
- 3. Check all constraints with the exams already assigned. If consistent, move to the next exam.
- 4. If no value is consistent for some exam, backtrack to the previous exam and try its next value.
- 5. Stop when all exams are assigned (solution) or all options are exhausted (no timetable).

Example: assign Maths = Day1-AM. For Physics (shares students with Maths) Day1-AM is removed; try Day1-PM. If later Chemistry has no legal slot, undo the last assignment and try another. Plain backtracking may waste time, discovering failure only deep in the tree, and has worst-case time O(d^n).

(iv) Constraint propagation and forward checking

Forward checking: after assigning a value to an exam, remove the inconsistent values from the domains of its unassigned neighbours. If any domain becomes empty, backtrack immediately.

Example: after setting Maths = Day1-AM, forward checking removes Day1-AM from Physics, Chemistry and all other exams that share students with Maths. If Physics then has no remaining slot, the failure is detected at once, instead of after many more assignments.

- Benefits: early failure detection, pruning of large parts of the search tree, fewer constraint checks later, and better combination with MRV (domain sizes are always up to date).
- Stronger propagation: arc consistency (AC-3) removes values that cannot satisfy any binary constraint, and Maintaining Arc Consistency (MAC) does so during search. It gives more pruning at extra cost per node.

(v) Real-world challenges and best strategy

- **Challenges:** huge numbers of courses/students; many conflicting and over-constrained requirements (may have no perfect solution); last-minute changes (cancellations, new accommodations); mixing hard and soft constraints; hall capacity and travel time between venues; fairness and student wellbeing.

Best strategy: use backtracking search enhanced with MRV + degree heuristic (choose the most constrained exam first), LCV for value ordering, forward checking / MAC for propagation, and intelligent backjumping (conflict-directed). For very large instances, add a local-search phase (min-conflicts) or Simulated Annealing to repair a near-complete timetable and optimise soft constraints, and decompose the problem by department/day to scale. For changes, repair the existing timetable with min-conflicts rather than re-solving from scratch. This combination guarantees all hard constraints, scales well, and handles soft preferences.

5. **Scenario:** Strategic Game AI for Real-Time Decision Making (Minimax & Alpha-Beta Pruning) A gaming company is developing an AI opponent for a real-time strategy game. The AI must complete against human players, Make decisions within strict time limits, Evaluate complex game states with multiple possible moves, The decision tree is extremely large, making computation expensive.

Tasks:

i. Explain how the Minimax algorithm models this problem

ii. Analyze the computational challenges of large game trees

iii. Explain how Alpha-Beta pruning improves efficiency with detailed reasoning

iv. Design an evaluation function and justify the factors considered

v. Recommend a strategy for balancing optimality and real-time performance

Answer 5: Real-Time Strategy Game AI – Minimax and Alpha-Beta Pruning

(i) Minimax model

Minimax models the game as a two-player, zero-sum adversarial search. The AI is MAX and tries to maximise its utility, while the human is MIN and tries to minimise it. The game tree has nodes = game states, edges = legal moves (alternating plies), and leaves = terminal states (win/lose/draw) or states at the depth limit scored by an evaluation function. Values are propagated upwards: a MAX node takes the maximum of its children, a MIN node the minimum. At the root, the AI picks the move with the highest minimax value, which is the best result it can guarantee against an optimal opponent.

Level	Node type	Child values	Node value
Leaf	Terminal / evaluated	3, 5, 2, 9	–
MIN node 1	MIN	3, 5	3
MIN node 2	MIN	2, 9	2
Root	MAX	3, 2	3 (choose MIN node 1)

(ii) Computational challenges

- **Exponential growth:** with branching factor b and depth d, the tree has O(b^d) nodes; time is O(b^d) and space O(b·d) (depth first). Example: b = 30 and d = 6 gives 30^6 ≈ 7.3 × 10^8 nodes; strategy games with hundreds of units give a much higher b.
- **Real-time limits:** decisions must be made within a fraction of a second while rendering/physics also consume CPU.
- **Complex states:** states include positions, resources, fog of war, simultaneous actions – expensive to generate and evaluate; imperfect information and randomness make pure minimax inexact.
- **Repeated states and poor move ordering** cause redundant work; the horizon effect appears when search is cut off.

(iii) How Alpha-Beta pruning improves efficiency

Alpha-Beta keeps two bounds during depth-first minimax: **α** = best (highest) value MAX can already guarantee on the path so far, **β** = best (lowest) value MIN can guarantee. A

branch is pruned (cut off) as soon as **α ≥ β**, because the opponent would never allow that position, so the remaining children cannot influence the final decision.

Example: Root (MAX) → MIN node A with leaves 3 and 5 gives value 3, so α = 3 at the root. MIN node B examines first leaf 2, so β = 2 ≤ α = 3 – MIN can force a value ≤ 2, worse than 3 for MAX. The remaining leaf of B (e.g. 9) is pruned without evaluation. The answer (3) is the same as minimax.

- **Reasoning:** the result is identical to Minimax (no loss of optimality) but irrelevant subtrees are skipped.
- **Best case (perfect move ordering):** O(b^(d/2)) – the search can go twice as deep for the same time, or the number of nodes is roughly the square root of minimax (effective branching factor ≈ √b).
- **Worst case (bad ordering):** same as minimax, O(b^d). Hence move ordering is essential: examine captures, threats and previous best moves first (killer-move / history heuristics, iterative deepening).
- Further tools: transposition tables (avoid re-searching repeated states) and aspiration windows.

(iv) Evaluation function design

Used at depth cutoff to estimate the utility of a non-terminal state, it is a weighted linear function: **Eval(s) = w1·f1 + w2·f2 + … + wn·fn**. It must be fast to compute, correlated with the real chance of winning, and the same function sign for both players.

Feature	Description	Why it matters	Example weight
Material / army strength	(Own units value − enemy units value), weighted by unit type	Stronger army wins battles	w1 = 5
Resources and economy	Gold/resources, worker count, production rate	Supports future production and upgrades	w2 = 3
Territory / map control	Controlled bases, chokepoints, vision	Gives safe expansion and information	w3 = 2
Tech / upgrades	Level of technology and unit upgrades	Long-term strength	w4 = 1.5
Position and safety	Unit positioning, base defence, threat to key buildings	Prevents sudden loss	w5 = 2
Mobility / options	Number of available moves	More flexibility	w6 = 0.5
Win/Loss terminal	+∞ if enemy base destroyed, −∞ if own base lost	Overrides all other features	±1000

Weights are tuned from expert knowledge, self-play simulations or machine learning (e.g. regression / neural network on game outcomes). Features that are cheap but informative are preferred, and weights are checked so that the function does not mislead search (e.g. over-valuing material while losing the base).

(v) Strategy to balance optimality and real-time performance

- **Alpha-Beta with iterative deepening:** search depth 1, 2, 3 … until the time limit ends and use the best move from the last completed depth; this gives an answer at any time.
- **Depth-limited search + evaluation function:** cut off at a depth that fits the time budget; use quiescence search in unstable states (battles) to reduce the horizon effect.
- **Move ordering and pruning:** best-first ordering (killer/history heuristics), forward pruning of obviously bad moves (beam/ProbCut), and transposition tables.
- **Abstraction:** treat groups of units as single actions and plan at a high level (strategic) while a lower-level controller handles micro-management.
- **Parallel/time management:** allocate more time to critical positions, use opening books and precomputed responses, and run the search on a separate thread.
- **Alternatives for very large trees:** Monte-Carlo Tree Search (MCTS), which samples promising lines and also handles uncertainty.

Conclusion: Alpha-Beta pruning with iterative deepening, a good evaluation function and strong move ordering gives near-optimal decisions within strict real-time limits; MCTS or abstraction can be added when the branching factor is too large.

RUBRICS FOR CALCULATION

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand